

Spring MVC

Complete Study Notes — Telusko Learning

1. What is Spring MVC?

Spring MVC is the **web layer** of the Spring Framework. It follows the **MVC (Model-View-Controller)** pattern and is used to build web applications that serve HTML pages (using JSP / Thymeleaf) or REST APIs returning JSON/XML.

Spring Boot auto-configures the entire MVC layer — you just write controllers and views, and the framework wires everything together.

2. Request Lifecycle (The 5 Steps)

Every HTTP request passes through these five components in order:

Step	Component	What it does
1	DispatcherServlet (Front Controller)	Single entry point. Receives all HTTP requests and orchestrates the entire lifecycle.
2	HandlerMapping	Given a URL (e.g. /telusko1), finds which controller method should handle it.
3	Controller	Your Java class. Processes the request, builds model data, returns a logical view name.
4	ViewResolver	Translates the logical view name ("index") into an actual file path (/view/index.jsp).
5	View (JSP)	Merges model data into the template and generates the final HTML response.

Mental model: Browser → DispatcherServlet → HandlerMapping → Controller → ViewResolver → JSP → HTML back to browser.

3. Three Ways to Pass Data to the View

Your **FirstController** demonstrates three approaches — all achieve the same result of passing data to the JSP.

Method 1 — ModelAndView (telusko1, telusko2)

Most explicit approach. Create a ModelAndView object, add data, set view name, return it.

```
ModelAndView mav = new ModelAndView();

mav.addObject("message", "Hello!"); // adds to model

mav.setViewName("index");           // sets view name

return mav;
```

Method 2 — Model parameter (telusko3)

Spring injects a Model object. Add attributes to it, return just the view name as a String.

```
public String displayInfo3(Model model) {

    model.addAttribute("message", "IPL Match KKR vs LSG");

    return "index"; // return only the view name

}
```

Internally Spring uses BindingAwareModelMap — you can verify with `model.getClass().getName()` as shown in the code.

Method 3 — Raw Map (telusko4)

Spring injects a plain Map. Least common, but valid. Spring wraps it into the model automatically.

```
public String displayInfo4(Map map) {

    map.put("message", "Learning during IPL match");

    return "index";

}
```

4. HTTP Method Annotations

Use specific mapping annotations instead of the generic `@RequestMapping` for clarity:

Annotation	HTTP Verb	Common Use
<code>@RequestMapping</code>	Any (GET by default)	Legacy / flexible catch-all
<code>@GetMapping</code>	GET	Loading pages, fetching data
<code>@PostMapping</code>	POST	Form submissions, creating records
<code>@PutMapping</code>	PUT	Full updates
<code>@PatchMapping</code>	PATCH	Partial updates
<code>@DeleteMapping</code>	DELETE	Deleting records

*Prefer `@GetMapping`, `@PostMapping`, etc. over `@RequestMapping` for readability.
`@RequestMapping("/telusko1")` is equivalent to `@GetMapping("/telusko1")` for GET requests.*

5. ViewResolver & application.properties

The ViewResolver translates a logical view name (a String like **"index"**) into the actual file path. This is configured in **application.properties**:

```
spring.application.name=web-app-1

server.port=8484           # run on port 8484, not default 8080

spring.mvc.view.prefix=/view/ # JSP files live in src/main/webapp/view/

spring.mvc.view.suffix=.jsp  # file extension is .jsp
```

So when a controller returns **"index"**:

- prefix + name + suffix = **/view/ + index + .jsp**
- Final resolved path: **/view/index.jsp**

Without prefix/suffix config, returning "index" would resolve to nothing and cause a 404.

6. The JSP View File

The JSP file receives model attributes set by the controller. There are two ways to print a model attribute in a JSP:

JSP Expression (legacy)

```
<%= request.getAttribute("message") %>
```

EL (Expression Language) — preferred

```
${message}
```

The DispatcherServlet automatically sets model attributes as **request attributes** before forwarding to the JSP — that is why both approaches work.

7. Spring MVC vs RESTful Services

Spring supports two usage modes, as shown in the whiteboard notes:

Mode	Returns	Annotations used	View Layer
Spring WebMVC (Traditional)	HTML pages	@Controller @RequestMapping	JSP / Thymeleaf
RESTful Services	JSON / XML	@RestController @GetMapping etc.	No view (data only)

8. Quick Reference — Key Points to Remember

<code>DispatcherServlet</code>	Front Controller — handles ALL requests. Auto-configured by Spring Boot.
<code>@Controller</code>	Marks a class as a Spring MVC controller. Returns view names.
<code>@RestController</code>	@Controller + @ResponseBody combined. Returns data (JSON), not views.
<code>ModelAndView</code>	Object holding both the model map and the logical view name.
<code>Model / Map</code>	Simpler alternatives for passing data; return view name as a String.
<code>ViewResolver</code>	Converts logical view name to physical file using prefix + suffix.
<code>@GetMapping</code>	Handles GET requests. Use for reading / loading pages.
<code>@PostMapping</code>	Handles POST requests. Use for form submissions.
<code>model.addAttribute</code>	Adds key-value pair to model, accessible in view as <code>\${key}</code> .

